
7. Classification

Knowledge Discovery in Databases with Exercises

Dominik Probst, dominik.probst@fau.de

Computer Science 6 (Data Management), Friedrich-Alexander-Universität Erlangen-Nürnberg

Summer semester 2026

1. Basic Concepts

2. Decision Tree Induction

- Information Gain (ID3)

- Gain Ratio (C4.5)

- Gini Index (CART)

3. Rule-Based Classification

4. Bayes Classification Methods

5. Model Evaluation

- Evaluation Metrics

- Evaluation Strategies

6. Ensemble Methods

7. Summary

Basic Concepts

Supervised Learning

- The **training data** (observations, measurements, etc.) are accompanied by **labels** indicating the **class** of the observations.
- New data is classified based on a **model** created from the training data.

Unsupervised Learning

- Class labels of training data are unknown. Or rather, there are no training data.
- Given a set of measurements, observations, etc., the goal is to find classes or clusters in the data.
→ Clustering.

- **Classification:**

- Predicts **categorical class labels** (discrete, nominal).
- Constructs a model based on the training set and the values (class labels) in a classifying attribute and uses it in classifying new data.

- **Numerical prediction:**

- Models **continuous-valued functions**.
- I.e. predicts missing or unknown (future) values.

- **Typical applications of classification:**

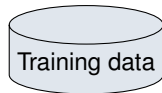
- Credit/loan approval: Will it be paid back?
- Medical diagnosis: Is a tumor cancerous or benign?
- Fraud detection: Is a transaction fraudulent or not?
- Web-page categorization: Which category is it such as to categorize it by topic.

1. Model construction: describing a set of predetermined classes:

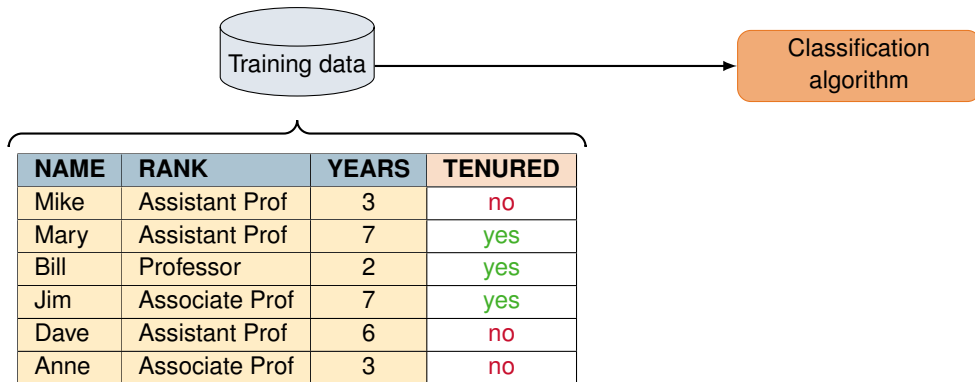
- Each tuple/sample is assumed to belong to a predefined class, as determined by the **class-label attribute**.
- The set of tuples used for model construction is the **training set**.
- The **model** is represented as classification rules, decision trees, or mathematical formulae.

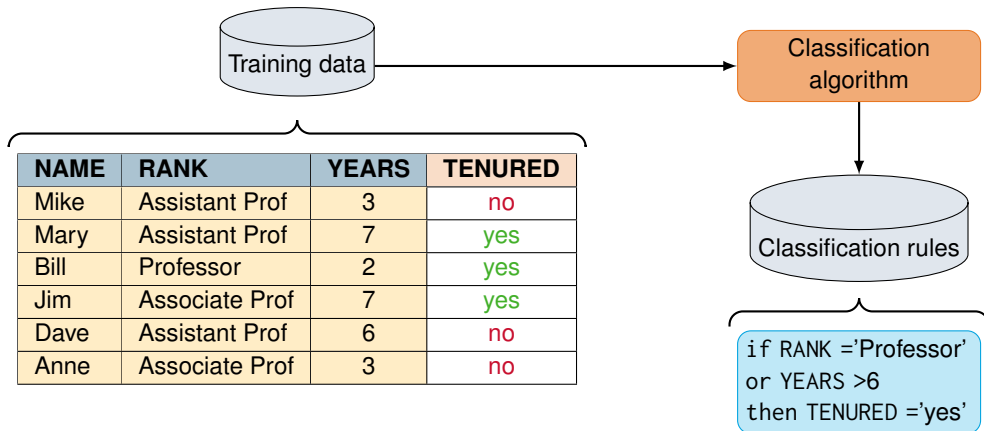
2. Model usage, for classifying future or unknown objects:

- Estimate **accuracy** of the model:
 - The known label of **test samples** is compared with the result from the model.
 - **Accuracy rate** is the percentage of test-set samples that are correctly classified by the model.
 - Test set is independent of training set (otherwise overfitting).
 - More on evaluation metrics later.
- If the accuracy is acceptable, **use the model** to classify data tuples whose class labels are not known.

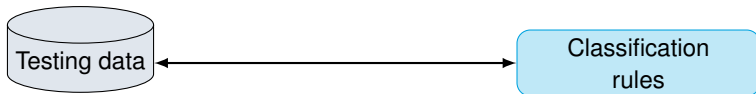


NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

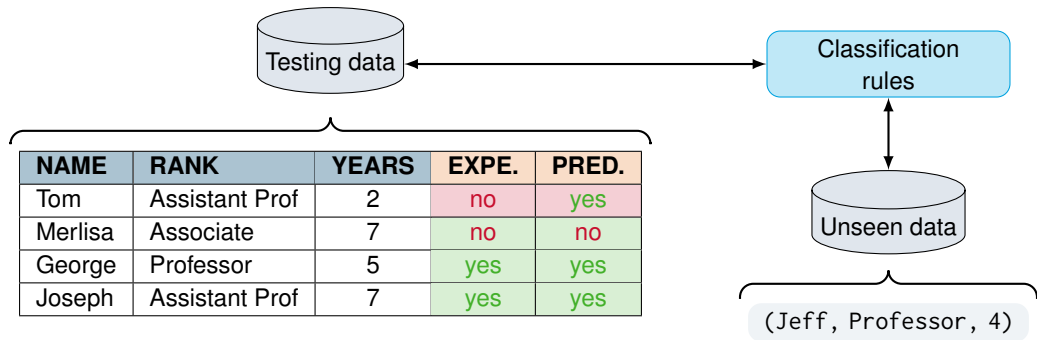


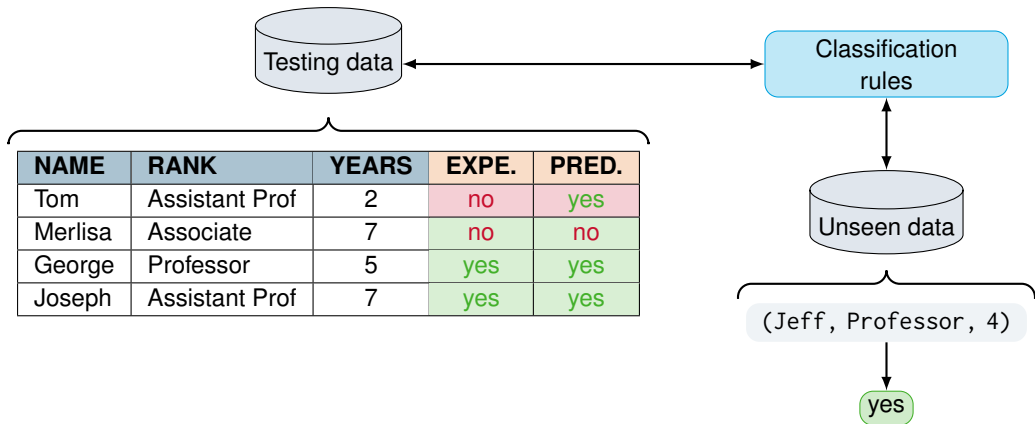


Classification
rules



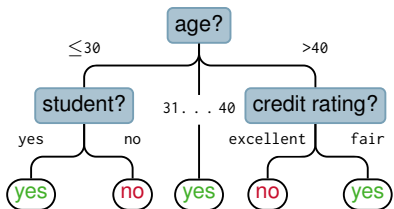
NAME	RANK	YEARS	EXPE.	PRED.
Tom	Assistant Prof	2	no	yes
Merlisa	Associate	7	no	no
George	Professor	5	yes	yes
Joseph	Assistant Prof	7	yes	yes





Decision Tree Induction

- **Training dataset:** buys_computer.
- **Resulting tree:**



age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

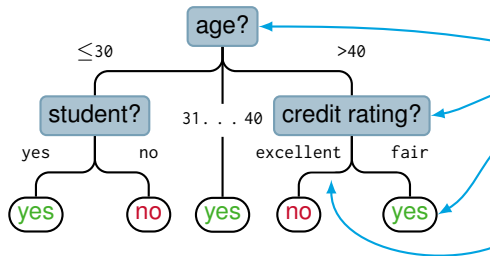
Decision Tree Induction

Decision tree induction refers to the learning of a decision-tree based on labeled training data.

Decision Tree

A *decision tree* is a flowchart-like structure consisting of interconnected internal and leaf nodes.

Components of a Decision Tree



- **Root:** topmost node.
- **Internal node:** test on an attribute.
- **Leaf node:** holds a class label, also called *terminal node*.
- **Branch:** outcome of a leaf node's test coupled with a text. In this example: excellent.

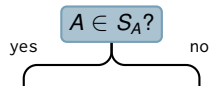
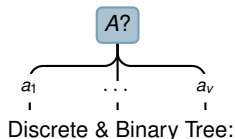
Construction in a *top-down recursive* and *divide-and-conquer* manner.

Input: data partition D , `attribute_list`,
`attribute_selection_method`.

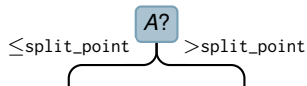
Algorithm Sketch `build_decision_tree`:

1. Create node N .
2. Determine splitting attribute A with `attribute_selection_method`.
3. Label N with splitting criterion.
4. If the splitting attribute has been fully utilized, remove it from `attribute_list`.
5. For each outcome of splitting criterion:
 - Partition D according to outcome of splitting criterion.
 - Grow branches on N for each partition.
6. Return node N

Attribute Types:
Discrete:



Continuous:



Stopping criteria:

- All samples in D belong to the same class:
⇒ N becomes a leaf.
- Samples in D belong to multiple classes, but `attribute_list` is empty:
⇒ Create leaf with majority class.
- Partition D is empty:
⇒ Create leaf with majority class.

Decision Tree Algorithm

A detailed algorithm to construct a decision tree is covered in the appendix and in our reference book¹.

¹J. Han et al., *Data Mining: Concepts and Techniques*, 3rd. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011, ISBN: 0123814790, pp. 332 – 335.

Attribute Selection Methods

An *attribute selection method* is a heuristic to determine the “best” splitting criterion to partition data.

- Also known as *splitting rules*.
- Provides ranking for each attribute.
- Partition data based on attribute with best score.
- Tree node is labeled with splitting criterion (attribute).

We will discuss **three** popular attribute selection methods²:

1. Information Gain
2. Gain Ratio
3. Gini Index

²Since this is not even close to an exhaustive list, we list some other popular methods in the appendix.

Decision Tree Induction

Information Gain (ID3)

- Partitions reflect least randomness, i. e. impurity.
- **Expected information** (entropy) needed to classify a tuple in D :

$$\text{Info}(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

- p_i is the probability that tuple in D belongs to class C_i , estimated by $\frac{|C_i|}{|D|}$.
- $\log_2$ because information is encoded in bits.

Calculate information for every attribute in `attribute_list` and data partition D :

Calculate information for every attribute in `attribute_list` and data partition D :

Discrete-valued Attribute

- Attribute A with v distinct values.
- Expected information required to classify tuple in D based on partitioning by A .
- D_A : dataset D partitioned by A ,
 $D_{A,j}$: dataset D partitioned by A with distinct value j .

$$\text{Info}_A(D) = \sum_{j=1}^v \frac{|D_{A,j}|}{|D_A|} \text{Info}(D_{A,j})$$

Calculate information for every attribute in `attribute_list` and data partition D :

Discrete-valued Attribute

- Attribute A with v distinct values.
- Expected information required to classify tuple in D based on partitioning by A .
- D_A : dataset D partitioned by A ,
 $D_{A,j}$: dataset D partitioned by A with distinct value j .

$$\text{Info}_A(D) = \sum_{j=1}^v \frac{|D_{A,j}|}{|D_A|} \text{Info}(D_{A,j})$$

Continuous-valued Attribute

- Attribute A with v distinct values.
- Order values in increasing order.
- Calculate midpoint of every neighbouring value.
- $v - 1$ possible split points $s_i = \frac{a_i + a_{i+1}}{2}$.
- Evaluate $\text{Info}_A(D)$ for every possible binary splitting ($A \leq s_i$ and $A > s_i$).

$$\text{Info}_A(D) = \frac{|D_{A \leq s_i}|}{|D_A|} \text{Info}(D_{A \leq s_i}) + \frac{|D_{A > s_i}|}{|D_A|} \text{Info}(D_{A > s_i})$$

Given $\text{Info}(D)$ and $\text{Info}_A(D)$, Information Gain of each attribute A is calculated as:

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

Information Gain

The attribute with the **highest** Information Gain is selected as the splitting attribute.

- **Target attribute:** buys_computer
- **Expected Information:**

$$\begin{aligned}\text{Info}(D) &= I(9, 5) \\ &= -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) \\ &= 0.94\end{aligned}$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Attribute:** Age
- **Value distribution:**

Age	Yes	No	$I(\text{Yes}, \text{No})$
≤ 30	2	3	0.971
31 ... 40	4	0	0
> 40	3	2	0.971

- **Calculation:**

$$\text{Info}_{\text{age}}(D) = \frac{5}{14} I(2, 3) + \frac{4}{14} I(4, 0) + \frac{5}{14} I(3, 2) \\ = 0.694$$

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) \\ = 0.94 - 0.694 \\ = 0.246$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Attribute:** Age

- **Value distribution:**

Age	Yes	No	$I(\text{Yes}, \text{No})$
≤ 30	2	3	0.971
31 ... 40	4	0	0
> 40	3	2	0.971

- **Calculation:**

$$\text{Info}_{\text{age}}(D) = \frac{5}{14} I(2, 3) + \frac{4}{14} I(4, 0) + \frac{5}{14} I(3, 2) \\ = 0.694$$

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) \\ = 0.94 - 0.694 \\ = 0.246$$

- **Gain of other attributes:**

- $\text{Gain}(\text{income}) = 0.029,$
- $\text{Gain}(\text{student}) = 0.151,$
- $\text{Gain}(\text{credit_rating}) = 0.048.$

- **Attribute:** Age
- **Value distribution:**

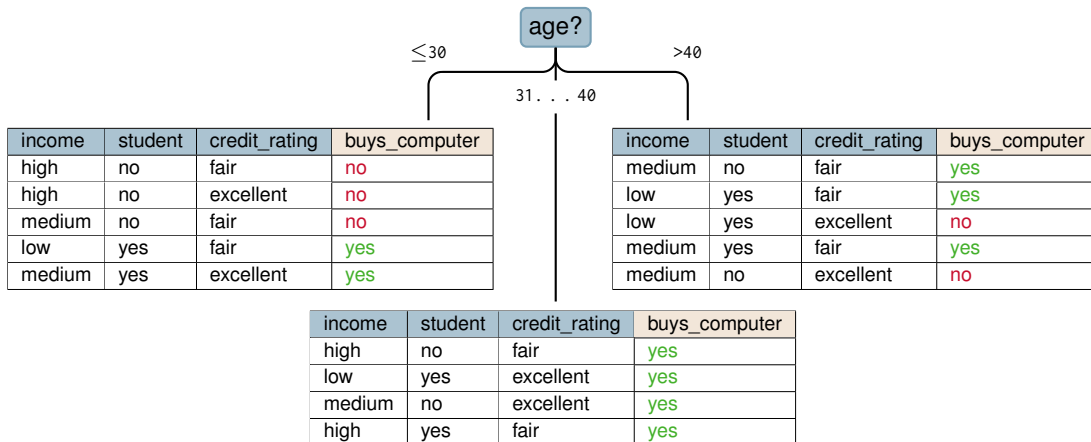
Age	Yes	No	$I(\text{Yes}, \text{No})$
≤ 30	2	3	0.971
31 ... 40	4	0	0
> 40	3	2	0.971

- **Calculation:**

$$\text{Info}_{\text{age}}(D) = \frac{5}{14} I(2, 3) + \frac{4}{14} I(4, 0) + \frac{5}{14} I(3, 2) \\ = 0.694$$

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) \\ = 0.94 - 0.694 \\ = 0.246$$

- **Gain of other attributes:**
 - $\text{Gain}(\text{income}) = 0.029$,
 - $\text{Gain}(\text{student}) = 0.151$,
 - $\text{Gain}(\text{credit_rating}) = 0.048$.
- **Splitting attribute:**
Age with $\text{Gain}(\text{age}) = 0.246$.



Decision Tree Induction

Gain Ratio (C4.5)

- **Problem of Information Gain:**

- Tends to prefer attributes with large number of distinct values.
 - E. g. attribute `degree_program` with 278 values vs. `student_status` with 2 values.

- **Problem of Information Gain:**

- Tends to prefer attributes with large number of distinct values.
 - E. g. attribute degree_program with 278 values vs. student_status with 2 values.

- **Solution:**

- Normalize the Information Gain to get the **Gain Ratio (C4.5)**:

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \log_2 \left(\frac{|D_j|}{|D|} \right)$$

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}$$

- **Disadvantage:** Becomes unstable as $\text{SplitInfo}_A(D)$ approaches zero.

Gain Ratio

The attribute with the **highest** Gain Ratio is selected as the splitting attribute.

- **Attribute:** Age

- **Calculation:**

$$\text{Gain}(\text{age}) = 0.246$$

$$\begin{aligned}\text{SplitInfo}_{\text{age}}(D) &= -\frac{5}{14} \log_2 \left(\frac{5}{14} \right) - \frac{4}{14} \log_2 \left(\frac{4}{14} \right) \\ &\quad - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \\ &= 1.577\end{aligned}$$

$$\begin{aligned}\text{GainRatio}(\text{age}) &= \frac{0.246}{1.577} \\ &= 0.156\end{aligned}$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

Decision Tree Induction

Gini Index (CART)

- **Problem of Information Gain and Gain Ratio:**
 - Use of logarithm is computationally expensive.
- **Solution:**
 - Use the **Gini Index (CART)** as an alternative to Information Gain and Gain Ratio.
 - Measures the **statistical dispersion** of a dataset.

- **Measured impurity** of partition D is defined as the sum over n classes:

$$\text{Gini}(D) = 1 - \sum_{j=1}^n p_j^2$$

- p_j is the non-zero probability that sample in D belongs to class C_j as estimated by $\frac{|C_{j,D}|}{|D|}$

- **Measured impurity** of partition D is defined as the sum over n classes:

$$\text{Gini}(D) = 1 - \sum_{j=1}^n p_j^2$$

- p_j is the non-zero probability that sample in D belongs to class C_j as estimated by $\frac{|C_{j,D}|}{|D|}$

- **Measured impurity** of partition D is defined as the sum over n classes:

$$\text{Gini}(D) = 1 - \sum_{j=1}^n p_j^2$$

- p_j is the non-zero probability that sample in D belongs to class C_j as estimated by $\frac{|C_{j,D}|}{|D|}$

Discrete-valued Attribute

- Attribute A with v distinct values.
- Compute all possible subsets of values.
- Compute weighted sum of each partition tuple $(D_1$ and $D_2)$ as follows:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

- **Measured impurity** of partition D is defined as the sum over n classes:

$$\text{Gini}(D) = 1 - \sum_{j=1}^n p_j^2$$

- p_j is the non-zero probability that sample in D belongs to class C_j as estimated by $\frac{|C_{j,D}|}{|D|}$

Discrete-valued Attribute

- Attribute A with v distinct values.
- Compute all possible subsets of values.
- Compute weighted sum of each partition tuple (D_1 and D_2) as follows:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

Continuous-valued Attribute

- Order values in increasing order.
- Split the dataset at every midpoint.
- Evaluate $\text{Gini}_A(D)$ for every possible binary splitting:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

- The **reduction in impurity** is defined as follows:

$$\Delta \text{Gini}_A(D) = \text{Gini}(D) - \text{Gini}_A(D).$$

Gini Index

The attribute (and partitioning) with the **highest** reduction in impurity is selected as the splitting attribute.

- **Target attribute:** buys_computer
- **Measured impurity:**

$$\begin{aligned} \text{Gini}(D) &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\ &= 0.459 \end{aligned}$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Attribute:** Income

- **Subsets:**

$D_1 : \{\text{low, medium}\}$

$D_2 : \{\text{high}\}$

- **Calculation:**

$$\begin{aligned} \text{Gini}(D|_{i=\{\text{high}\}}) &= \text{Gini}(D|_{i=\{\text{medium, low}\}}) \\ &= \frac{10}{14} \text{Gini}(D_1) + \frac{4}{14} \text{Gini}(D_2) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) \\ &\quad + \frac{4}{14} \left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) \\ &= 0.443 \end{aligned}$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Attribute:** Income

- **Subsets:**

$$D_1 : \{\text{low}, \text{medium}\}$$

$$D_2 : \{\text{high}\}$$

- **Calculation:**

$$\begin{aligned} \text{Gini}(D|_{i=\{\text{high}\}}) &= \text{Gini}(D|_{i=\{\text{medium}, \text{low}\}}) \\ &= \frac{10}{14} \text{Gini}(D_1) + \frac{4}{14} \text{Gini}(D_2) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) \\ &\quad + \frac{4}{14} \left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) \\ &= 0.443 \end{aligned}$$

- **Reduction in impurity:**

$$\begin{aligned} \Delta \text{Gini}_{i=\{\text{high}\}}(D) &= \text{Gini}(D) - \text{Gini}(D|_{i=\{\text{high}\}}) \\ &= 0.459 - 0.443 \\ &= 0.016 \end{aligned}$$

- **Other subsets:**

- $\Delta \text{Gini}_{i=\{\text{medium}\}}(D) = 0.001$
- $\Delta \text{Gini}_{i=\{\text{low}\}}(D) = 0.009$

- **Attribute:** Income

- **Subsets:**

$$D_1 : \{\text{low}, \text{medium}\}$$

$$D_2 : \{\text{high}\}$$

- **Calculation:**

$$\begin{aligned} \text{Gini}(D|_{i=\{\text{high}\}}) &= \text{Gini}(D|_{i=\{\text{medium}, \text{low}\}}) \\ &= \frac{10}{14} \text{Gini}(D_1) + \frac{4}{14} \text{Gini}(D_2) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) \\ &\quad + \frac{4}{14} \left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) \\ &= 0.443 \end{aligned}$$

- **Reduction in impurity:**

$$\begin{aligned} \Delta \text{Gini}_{i=\{\text{high}\}}(D) &= \text{Gini}(D) - \text{Gini}(D|_{i=\{\text{high}\}}) \\ &= 0.459 - 0.443 \\ &= 0.016 \end{aligned}$$

- **Other subsets:**

- $\Delta \text{Gini}_{i=\{\text{medium}\}}(D) = 0.001$
- $\Delta \text{Gini}_{i=\{\text{low}\}}(D) = 0.009$

- **Splitting subset:**

- If income has the overall highest reduction of impurity, then the split is on {"low", "medium"} and {"high"}.

Decision Tree Induction

Overview

The three methods, in general, return good results, but

- **Information Gain:**
 - Biased towards multi-valued attributes.
- **Gain Ratio:**
 - Tends to prefer unbalanced splits in which one partition is much smaller than the others.
- **Gini Index:**
 - Biased to multi-valued attributes.
 - Has difficulty when number of classes is large.
 - Tends to favor tests that result in equal-sized partitions and purity in both partitions.

- **Problem:**

- Many branches may reflect anomalies due to noise or outliers.
- Overall poor accuracy for unseen samples.

⇒ **Overfitting:** An induced tree may overfit the training data.

- **Problem:**

- Many branches may reflect anomalies due to noise or outliers.
- Overall poor accuracy for unseen samples.

⇒ **Overfitting:** An induced tree may overfit the training data.

- **Solution:**

- **Pruning:** Avoid branches that have little importance.
- **Two** types of pruning:
 1. **Prepruning:** Stop growing the tree early.
 2. **Postpruning:** Remove branches from a "fully grown" tree.

- A lot of research has been done to improve basic **decision tree induction** algorithms. E.g.:
 - **Allow for continuous-valued attributes.**
 - Dynamically define new discrete-valued attributes that partition the values of continuous-valued attributes into a discrete set of intervals.
 - **Handle missing attribute values.**
 - Assign the most common value of the attribute.
 - Assign probability to each of the possible values.
 - **Attribute construction.**
 - Create new attributes based on existing ones that are sparsely represented.
 - This reduces fragmentation, repetition, and replication.

- **Problem:**

- Basic decision tree algorithms (ID3, C4.5, and CART) are not scalable (They require the entire dataset to be in memory).
- They are not designed to handle large datasets.

- **Solution:**

- Extend the basic algorithms to handle large datasets.
- We will take a look at **two** modifications/methods:
 1. RainForest
 2. BOAT

- **Basic Idea:**
 - Extract all data required for the attribute selection methods
⇒ Store it in a **compact** data structure.
 - Apply the original algorithm to the compact data structure(s).
- **Data structure(s):**
 - **AVC-list:**
 - Attribute
 - Value
 - Class label
 - **AVC-set (of an attribute X):**
 - Aggregated projection of training dataset onto the attribute X with counts of each class label.
 - **AVC-group (of a node n):**
 - Set of AVC-sets of all predictor attributes at the node n .

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

AVC-set on age:

age	yes	no
≤ 30	2	3
31 ... 40	4	0
> 40	3	2

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

AVC-set on age:

age	yes	no
≤ 30	2	3
31 ... 40	4	0
> 40	3	2

AVC-set on income:

income	yes	no
high	2	2
medium	4	2
low	3	1

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

AVC-set on age:

age	yes	no
≤ 30	2	3
31 ... 40	4	0
> 40	3	2

AVC-set on income:

income	yes	no
high	2	2
medium	4	2
low	3	1

AVC-set on student:

student	yes	no
yes	6	1
no	3	4

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

AVC-set on age:

age	yes	no
≤ 30	2	3
31 ... 40	4	0
> 40	3	2

AVC-set on income:

income	yes	no
high	2	2
medium	4	2
low	3	1

AVC-set on student:

student	yes	no
yes	6	1
no	3	4

AVC-set on credit_rating:

credit_rating	yes	no
fair	6	2
excellent	3	3

- **Basic Idea:**

- Use a statistical technique to create several smaller samples (subsets).
- Every sample fits in the memory and is used to induce a decision tree.
- All trees are combined to form a single tree T' .

- **Advantages:**

- Resulting tree often very close to the tree induced from the entire dataset.
- Requires only two scans of the DB.
- An incremental algorithm:
 - Take insertions and deletions of training data and update the decision tree.

BOAT

The full title of BOAT is *Bootstrapped Optimistic Algorithm for Tree Construction*, which refers to the underlying statistical technique used: **Bootstrapping**.

Rule-Based Classification

- Rule-based classification is based on a set of **IF-THEN rules**.
- Each **IF-THEN rule** consists of two parts:
 - **IF** (antecedent/precondition): a condition or set of conditions that must be satisfied.
 - **THEN** (consequent): the conclusion or action that follows if the IF part is satisfied.

- Rule-based classification is based on a set of **IF-THEN rules**.
- Each **IF-THEN rule** consists of two parts:
 - **IF** (antecedent/precondition): a condition or set of conditions that must be satisfied.
 - **THEN** (consequent): the conclusion or action that follows if the IF part is satisfied.
- **Example:** (one rule)
 - **IF** age $\leq$ 30 AND student = "yes" **THEN** buys_computer = "yes".

- Rule-based classification is based on a set of **IF-THEN rules**.
- Each **IF-THEN rule** consists of two parts:
 - **IF** (antecedent/precondition): a condition or set of conditions that must be satisfied.
 - **THEN** (consequent): the conclusion or action that follows if the IF part is satisfied.
- **Example:** (one rule)
 - **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"yes"}$ **THEN** $\text{buys_computer} = \text{"yes"}$.
- Very **easy** to read and understand for humans.

- Given is a **set of rules**:
 - **IF** $\text{price} < 1500$ **THEN** $\text{buy} = \text{"yes"}$.
 - **IF** $\text{price} \geq 1500$ **AND** $\text{color} = \text{"red"}$ **THEN** $\text{buy} = \text{"no"}$.
 - **IF** $\text{price} \geq 1500$ **AND** $\text{location} = \text{"Erlangen"}$ **THEN** $\text{buy} = \text{"yes"}$.

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	?
2306	red	Erlangen	?
1995	green	Fuerth	?

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	?
2306	red	Erlangen	?
1995	green	Fuerth	?

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	yes
2306	red	Erlangen	?
1995	green	Fuerth	?

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	yes
2306	red	Erlangen	?
1995	green	Fuerth	?

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	yes
2306	red	Erlangen	?
1995	green	Fuerth	?

- Some scenarios might lead to **conflicts**:
 - More than one rule is triggered.

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	yes
2306	red	Erlangen	?
1995	green	Fuerth	?

- Some scenarios might lead to **conflicts**:
 - More than one rule is triggered.

- Given is a **set of rules**:
 - IF** price < 1500 **THEN** buy = "yes".
 - IF** price $\geq$ 1500 AND color = "red" **THEN** buy = "no".
 - IF** price $\geq$ 1500 AND location = "Erlangen" **THEN** buy = "yes".
- The set may be used to classify **new tuples**:

price	color	location	buy
1349	red	Nuremberg	yes
2306	red	Erlangen	?
1995	green	Fuerth	?

- Some scenarios might lead to **conflicts**:
 - More than one rule is triggered.
 - No rule is triggered.

1. More than one rule is triggered: **conflict resolution**.

- **Size ordering:**
 - Assign the highest priority to the triggered rule that has the "toughest" requirement (i.e., rule with most used attribute in condition).
- **Class-based ordering:**
 - Decreasing order of prevalence or misclassification cost per class.
 - No order of rules within class → disjunction (logical OR) between rules.
- **Rule-based ordering** (decision list):
 - Rules are organized into one long priority list, according to some measure of rule quality, or by experts.
 - Rules must be applied in this particular order to avoid conflict.

1. More than one rule is triggered: **conflict resolution**.

- **Size ordering:**
 - Assign the highest priority to the triggered rule that has the "toughest" requirement (i.e., rule with most used attribute in condition).
- **Class-based ordering:**
 - Decreasing order of prevalence or misclassification cost per class.
 - No order of rules within class → disjunction (logical OR) between rules.
- **Rule-based ordering** (decision list):
 - Rules are organized into one long priority list, according to some measure of rule quality, or by experts.
 - Rules must be applied in this particular order to avoid conflict.

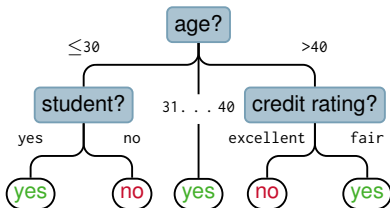
2. No rule is triggered.

- Use a fallback/default rule.
- Always evaluated as the last rule, if and only if other rules are not covered by some tuple, i. e. no rules have been triggered.

- **Basic idea:**
 - Rules are **easier to understand** than large trees.
 - A rule can be created for **each path from the root to a leaf**.
 - Each attribute-value pair along the path forms **a condition**:

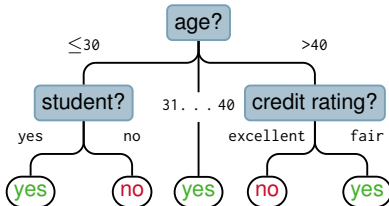
- **Basic idea:**

- Rules are **easier to understand** than large trees.
- A rule can be created for **each path from the root to a leaf**.
- Each attribute-value pair along the path forms **a condition**:



- **Basic idea:**

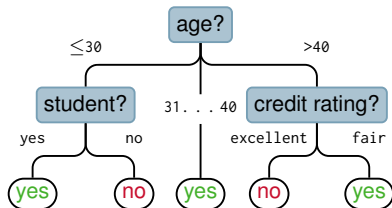
- Rules are **easier to understand** than large trees.
- A rule can be created for **each path from the root to a leaf**.
- Each attribute-value pair along the path forms **a condition**:



1. **IF** age ≤ 30 AND student = "yes"
THEN buys_computer = "yes".

- **Basic idea:**

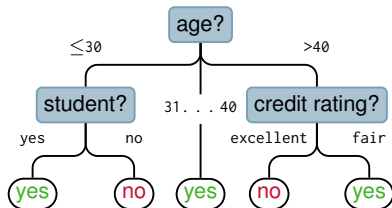
- Rules are **easier to understand** than large trees.
- A rule can be created for **each path from the root to a leaf**.
- Each attribute-value pair along the path forms **a condition**:



1. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"yes"}$
THEN $\text{buys_computer} = \text{"yes"}$.
2. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"no"}$
THEN $\text{buys_computer} = \text{"no"}$.

- **Basic idea:**

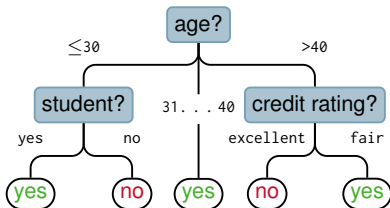
- Rules are **easier to understand** than large trees.
- A rule can be created for **each path from the root to a leaf**.
- Each attribute-value pair along the path forms **a condition**:



1. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"yes"}$
THEN $\text{buys_computer} = \text{"yes"}$.
2. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"no"}$
THEN $\text{buys_computer} = \text{"no"}$.
3. **IF** $\text{age} = 31 \dots 40$
THEN $\text{buys_computer} = \text{"yes"}$.

- **Basic idea:**

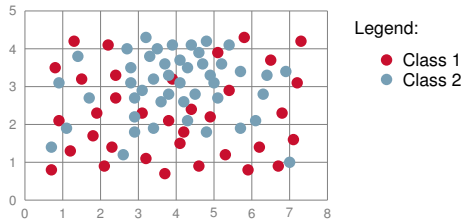
- Rules are **easier to understand** than large trees.
- A rule can be created for **each path from the root to a leaf**.
- Each attribute-value pair along the path forms **a condition**:



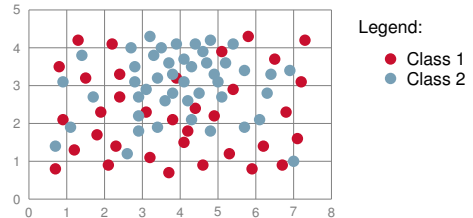
1. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"yes"}$
THEN $\text{buys_computer} = \text{"yes"}$.
2. **IF** $\text{age} \leq 30$ AND $\text{student} = \text{"no"}$
THEN $\text{buys_computer} = \text{"no"}$.
3. **IF** $\text{age} = 31 \dots 40$
THEN $\text{buys_computer} = \text{"yes"}$.
4. ...

- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.

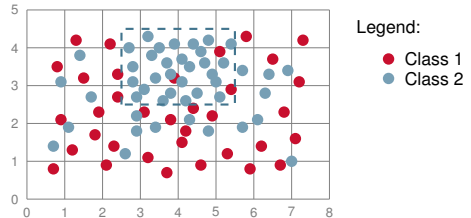
- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.
- **Steps of the method:**



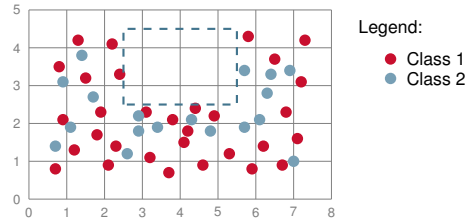
- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.
- **Steps of the method:**
 1. Start with an empty set of rules.



- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.
- **Steps of the method:**
 1. Start with an empty set of rules.
 2. Find the rule r with the best covering.



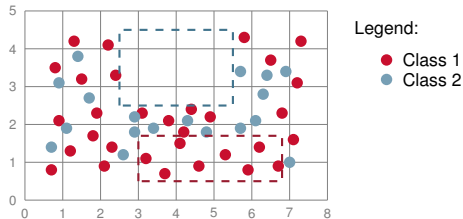
- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.
- **Steps of the method:**
 1. Start with an empty set of rules.
 2. Find the rule r with the best covering.
 3. Remove all tuples covered.



- Extracting rules from decision trees is not **the only** way to learn rules.
- Rules can be learned **directly** from **the training data**:
 - Rules are learned **sequentially**.
 - Each rule is optimized to cover **as many tuples of a given class** as possible while covering **as few tuples of other classes** as possible.

- **Steps of the method:**

1. Start with an empty set of rules.
2. Find the rule r with the best covering.
3. Remove all tuples covered.
4. Repeat with step 2 until:
 - No more tuples left.
 - The quality of a rule is below a threshold.



- **Typical sequential covering algorithms:**
 - FOIL³, AQ⁴, CN2⁵, RIPPER⁶.

³J. R. Quinlan, "Learning logical definitions from relations," *Mach. Learn.*, vol. 5, pp. 239–266, 1990. DOI: 10.1007/BF00117105 [Online]. Available: <https://doi.org/10.1007/BF00117105>

⁴R. S. Michalski et al., "The multi-purpose incremental learning system aq15 and its testing application to three medical domains," in *Proc. AAAI*, vol. 1986, 1986, pp. 1–041

⁵P. Clark and T. Niblett, "The cn2 induction algorithm," *Machine learning*, vol. 3, no. 4, pp. 261–283, 1989

⁶W. W. Cohen et al., "Fast effective rule induction," in *Proceedings of the twelfth international conference on machine learning*, 1995, pp. 115–123

- **Typical sequential covering algorithms:**

- FOIL, AQ, CN2, RIPPER.

- **FOIL (First-Order Inductive Learner):**

- Based on Information Gain
- Suppose we have two rules:

R : IF condition THEN class = c

R' : IF condition' THEN class = c

- pos/neg are # of positive/negative tuples covered by R , pos'/neg' respectively for R' .
- FOIL assesses the information gained by extending $condition'$ as

$$\text{FOIL_Gain} = pos' \left(\log_2 \frac{pos'}{pos' + neg'} - \log_2 \frac{pos}{pos + neg} \right).$$

- FOIL favors rules that have high accuracy and cover many positive tuples.

Bayes Classification Methods

- **Bayesian Classification:**
 - Statistical classification method.
 - Predict class membership probabilities.
 - Based on **Bayes' Theorem**.

³T. Bayes, "An essay towards solving a problem in the doctrine of chances," *Phil. Trans. of the Royal Soc. of London*, vol. 53, pp. 370–418, 1763

- **Bayesian Classification:**
 - Statistical classification method.
 - Predict class membership probabilities.
 - Based on **Bayes' Theorem**.

Bayes' Theorem³

Bayes' Theorem describes the probability of an event based on prior knowledge of conditions that might be related to the event.

³T. Bayes, "An essay towards solving a problem in the doctrine of chances," *Phil. Trans. of the Royal Soc. of London*, vol. 53, pp. 370–418, 1763

- **Bayesian Classification:**
 - Statistical classification method.
 - Predict class membership probabilities.
 - Based on **Bayes' Theorem**.
- **Our Focus:**
 - **Naïve Bayesian Classification**
 - Assumes conditional independence of attributes ("naïve").
 - Simplification of Bayesian classification.

Bayes' Theorem³

Bayes' Theorem describes the probability of an event based on prior knowledge of conditions that might be related to the event.

³T. Bayes, "An essay towards solving a problem in the doctrine of chances," *Phil. Trans. of the Royal Soc. of London*, vol. 53, pp. 370–418, 1763

- **Let ...**
 - ... X be a **data sample** ("evidence").
 - The class label shall be unknown.
 - ... C_i be the **hypothesis** that X belongs to class i .

- **Let ...**
 - ... X be a **data sample** ("evidence").
 - The class label shall be unknown.
 - ... C_i be the **hypothesis** that X belongs to class i .
- **Then our goal is to determine the Posteriori Probability $P(C_i|X)$:**
 - The probability that the hypothesis holds given the observed data sample X .

- **Let ...**
 - ... X be a **data sample** ("evidence").
 - The class label shall be unknown.
 - ... C_i be the **hypothesis** that X belongs to class i .
- **Then our goal is to determine the Posteriori Probability $P(C_i|X)$:**
 - The probability that the hypothesis holds given the observed data sample X .
- **To determine $P(C_i|X)$, we need to know:**
 - **The Prior Probability $P(C_i)$:**
 - The overall probability of the class i .
 - E.g., X will buy computer, regardless of age, income,
 - **The Likelihood $P(X|C_j)$:**
 - The probability of observing the sample X given that the hypothesis holds.
 - E.g., given that X buys computer, the probability that X is 31 . . . 40, medium income.
 - **The Probability of the sample data $P(X)$:**
 - The probability that sample data is observed.

- The **Posteriori Probability** $P(C_i|X)$ follows from the **Bayes' Theorem**:

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

- The **Posteriori Probability** $P(C_i|X)$ follows from the **Bayes' Theorem**:

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

The Maximum Posteriori Probability

Since the posteriori probability $P(C_i|X)$ is basically the probability that X belongs to class C_i , we want to find the class C_i that **maximizes** this probability. X **is classified** as belonging to this class.

- The **Posteriori Probability** $P(C_i|X)$ follows from the Bayes' Theorem:

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

The Maximum Posteriori Probability

Since the posteriori probability $P(C_i|X)$ is basically the probability that X belongs to class C_i , we want to find the class C_i that **maximizes** this probability. X **is classified** as belonging to this class.

- Since $P(X)$ is constant for all classes, we only have to maximize:

$$P(X|C_i)P(C_i).$$

- **Naïve Bayesian Classification:**

- **Assumption:** All attributes are *conditionally independent*.
- I.e. no dependence relation between attributes (which is "naïve").

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i) = P(x_1|C_i)P(x_2|C_i) \cdots P(x_n|C_i).$$

- Greatly reduces the computation cost.

- **Naïve Bayesian Classification:**

- **Assumption:** All attributes are *conditionally independent*.
- I.e. no dependence relation between attributes (which is "naïve").

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i) = P(x_1|C_i)P(x_2|C_i) \cdots P(x_n|C_i).$$

- Greatly reduces the computation cost.

- **Naïve Bayesian Classification:**

- **Assumption:** All attributes are *conditionally independent*.
- I.e. no dependence relation between attributes (which is "naïve").

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i) = P(x_1|C_i)P(x_2|C_i) \cdots P(x_n|C_i).$$

- Greatly reduces the computation cost.

Categorical Attribute

- $P(x_k|C_i)$ is the number of tuples in C_i having value x_k for A_k divided by $|C_{i,D}|$ (the number of tuples of C_i in D):

$$P(x_k|C_i) = \frac{|\{t \in C_i : t.A_k = x_k\}|}{|C_{i,D}|}$$

- **Naïve Bayesian Classification:**

- **Assumption:** All attributes are *conditionally independent*.
- I.e. no dependence relation between attributes (which is "naïve").

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i) = P(x_1|C_i)P(x_2|C_i) \cdots P(x_n|C_i).$$

- Greatly reduces the computation cost.

Categorical Attribute

- $P(x_k|C_i)$ is the number of tuples in C_i having value x_k for A_k divided by $|C_{i,D}|$ (the number of tuples of C_i in D):

$$P(x_k|C_i) = \frac{|\{t \in C_i : t.A_k = x_k\}|}{|C_{i,D}|}$$

Continuous-valued Attribute

- $P(x_k|C_i)$ is usually computed based on Gaussian distribution with a mean μ and standard deviation σ :

$$G(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$
$$P(x_k|C_i) = G(x_k, \mu_{C_i}, \sigma_{C_i})$$

- **Target attribute:** buys_computer

- **Data sample:**

$X = (\text{age} \leq 30,$
 $\text{income} = \text{"medium"},$
 $\text{student} = \text{"yes"},$
 $\text{credit_rating} = \text{"fair"})$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Target attribute:** buys_computer

- **Data sample:**

$X = (\text{age} \leq 30,$
 $\text{income} = \text{"medium"},$
 $\text{student} = \text{"yes"},$
 $\text{credit_rating} = \text{"fair"})$

- **Prior Probability $P(C_i)$:**

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Target attribute:** buys_computer

- **Data sample:**

$X = (\text{age} \leq 30,$
 $\text{income} = \text{"medium"},$
 $\text{student} = \text{"yes"},$
 $\text{credit_rating} = \text{"fair"})$

- **Prior Probability $P(C_i)$:**

$P(\text{yes}) =$

$P(\text{no}) =$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Target attribute:** buys_computer

- **Data sample:**

$X = (\text{age} \leq 30,$
 $\text{income} = \text{"medium"},$
 $\text{student} = \text{"yes"},$
 $\text{credit_rating} = \text{"fair"})$

- **Prior Probability $P(C_i)$:**

$$P(\text{yes}) = \frac{9}{14} \approx 0.643$$

$$P(\text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- **Target attribute:** buys_computer

- **Data sample:**

$X = (\text{age} \leq 30,$
 $\text{income} = \text{"medium"},$
 $\text{student} = \text{"yes"},$
 $\text{credit_rating} = \text{"fair"})$

- **Prior Probability $P(C_i)$:**

$$P(\text{yes}) = \frac{9}{14} \approx 0.643$$

$$P(\text{no}) = \frac{5}{14} \approx 0.357$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) =$$

$$P(\text{age} \leq 30 | \text{no}) =$$

$$P(\text{income} = \text{"medium"} | \text{yes}) =$$

$$P(\text{income} = \text{"medium"} | \text{no}) =$$

$$P(\text{student} = \text{"yes"} | \text{yes}) =$$

$$P(\text{student} = \text{"yes"} | \text{no}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:**

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) =$$

$$P(\text{income} = \text{"medium"} | \text{yes}) =$$

$$P(\text{income} = \text{"medium"} | \text{no}) =$$

$$P(\text{student} = \text{"yes"} | \text{yes}) =$$

$$P(\text{student} = \text{"yes"} | \text{no}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) =$$

$$P(\text{income} = \text{"medium"} | \text{no}) =$$

$$P(\text{student} = \text{"yes"} | \text{yes}) =$$

$$P(\text{student} = \text{"yes"} | \text{no}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) =$$

$$P(\text{student} = \text{"yes"} | \text{yes}) =$$

$$P(\text{student} = \text{"yes"} | \text{no}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) =$$

$$P(\text{student} = \text{"yes"} | \text{no}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) =$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) =$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31 ... 40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31 ... 40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31 ... 40	medium	no	excellent	yes
31 ... 40	high	yes	fair	yes
> 40	medium	no	excellent	no

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X | C_i)$:

$$P(X | \text{yes}) =$$

$$P(X | \text{no}) =$$

- Likelihood(s) $P(X_k|C_i)$:

$$P(\text{age} \leq 30|\text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30|\text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"}|\text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"}|\text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"}|\text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"}|\text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"}|\text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"}|\text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X|C_i)$:

$$P(X|\text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X|\text{no}) =$$

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X | C_i)$:

$$P(X | \text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X | \text{no}) = 0.6 \cdot 0.4 \cdot 0.2 \cdot 0.4 = 0.019$$

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X | C_i)$:

$$P(X | \text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X | \text{no}) = 0.6 \cdot 0.4 \cdot 0.2 \cdot 0.4 = 0.019$$

- Calculate $P(X | C_i) \cdot P(C_i)^a$:

^aReminder: Calculating the numerator of the posterior probability is sufficient to classify X

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X | C_i)$:

$$P(X | \text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X | \text{no}) = 0.6 \cdot 0.4 \cdot 0.2 \cdot 0.4 = 0.019$$

- Calculate $P(X | C_i) \cdot P(C_i)^a$:

$$P(X | \text{yes}) \cdot P(\text{yes}) = 0.028.$$

$$P(X | \text{no}) \cdot P(\text{no}) = 0.007.$$

^aReminder: Calculating the numerator of the posterior probability is sufficient to classify X

- Likelihood(s) $P(X_k | C_i)$:

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- Likelihood(s) $P(X | C_i)$:

$$P(X | \text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X | \text{no}) = 0.6 \cdot 0.4 \cdot 0.2 \cdot 0.4 = 0.019$$

- Calculate $P(X | C_i) \cdot P(C_i)^a$:

$$P(X | \text{yes}) \cdot P(\text{yes}) = 0.028.$$

$$P(X | \text{no}) \cdot P(\text{no}) = 0.007.$$

- Classification Result:

^aReminder: Calculating the numerator of the posterior probability is sufficient to classify X

- **Likelihood(s) $P(X_k|C_i)$:**

$$P(\text{age} \leq 30 | \text{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} \leq 30 | \text{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{"medium"} | \text{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{"yes"} | \text{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{no}) = \frac{2}{5} = 0.4$$

- **Likelihood(s) $P(X|C_i)$:**

$$P(X | \text{yes}) = 0.222 \cdot 0.444 \cdot 0.667 \cdot 0.667 \approx 0.044$$

$$P(X | \text{no}) = 0.6 \cdot 0.4 \cdot 0.2 \cdot 0.4 = 0.019$$

- **Calculate $P(X|C_i) \cdot P(C_i)^a$:**

$$P(X | \text{yes}) \cdot P(\text{yes}) = 0.028.$$

$$P(X | \text{no}) \cdot P(\text{no}) = 0.007.$$

- **Classification Result:**

- X belongs to class C_1
(buys_computer = yes)

^aReminder: Calculating the numerator of the posterior probability is sufficient to classify X

Zero-Probability Problem

Our naïve Bayes classifier performs poorly if any of the conditional probabilities **any** of the conditional probabilities $P(x_k|C_i)$ is zero, as this causes the **entire product** of probabilities to become zero.

Zero-Probability Problem

Our naïve Bayes classifier performs poorly if any of the conditional probabilities **any** of the conditional probabilities $P(x_k|C_i)$ is zero, as this causes the **entire product** of probabilities to become zero.

- **Example:**
 - income = "low" (0 tuples), "medium" (990 tuples), "high" (10 tuples).

Zero-Probability Problem

Our naïve Bayes classifier performs poorly if any of the conditional probabilities **any** of the conditional probabilities $P(x_k|C_i)$ is zero, as this causes the **entire product** of probabilities to become zero.

- **Example:**
 - income = "low" (0 tuples), "medium" (990 tuples), "high" (10 tuples).
- **Solution:**
 - Use **Laplacian correction** (or Laplacian estimator):
 - Add 1 to each case:

$$P(\text{income} = \text{"low"}) = \frac{1}{1003}$$

$$P(\text{income} = \text{"medium"}) = \frac{991}{1003}$$

$$P(\text{income} = \text{"high"}) = \frac{11}{1003}$$

- **Advantages of the Naïve Bayes Classifier:**
 - Easy to implement.
 - Good results obtained in most of the cases.
- **Disadvantages of the Naïve Bayes Classifier:**
 - Assumption: class conditional independence, therefore loss of accuracy.
 - Practically, **dependencies** exist among most variables.
 - Cannot be modeled by Naïve Bayesian Classifier.
 - **How to deal with these dependencies?**
→ Bayesian Belief Networks (not part of KDD⁴).

⁴More info in the reference book (Chapter 9.1): J. Han et al., *Data Mining: Concepts and Techniques*, 3rd. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011, ISBN: 0123814790

Model Evaluation

- **Classification models might perform differently depending on the use case.**
⇒ **Model evaluation** is crucial to select the best model for a specific task.
- **Model evaluation can be split into two parts:**
 - **Evaluation metrics:**
What metric is important for the task?
 - **Evaluation strategies:**
How to tackle the evaluation? E.g. how to split the data into training and test sets?

Model Evaluation

Evaluation Metrics

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Confusion Matrix:**

- Summarizes the results of a classification model.
- Shows the number of correct and incorrect predictions for each class.
- **Correctly classified tuples:**
 - **True Positives (TP):** Positive tuples correctly classified as positive.
 - **True Negatives (TN):** Negative tuples correctly classified as negative.
- **Incorrectly classified tuples:**
 - **False Positives (FP):** Negative tuples incorrectly classified as positive.
 - **False Negatives (FN):** Positive tuples incorrectly classified as negative.

		<i>Predicted Class</i>		<i>Total</i>
		C_1	$\neg C_1$	
<i>True Class</i>	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
<i>Total</i>		P'	N'	P+N

- **Accuracy:**
- **Error Rate:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Accuracy:**
 - Percentage of correctly classified tuples.
- **Error Rate:**

$$\text{Accuracy} = \frac{TP + TN}{P + N}$$

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Accuracy:**

- Percentage of correctly classified tuples.

$$\text{Accuracy} = \frac{TP + TN}{P + N}$$

- **Error Rate:**

- Inverse of accuracy, i.e. percentage of incorrectly classified tuples.

$$\begin{aligned}\text{Error Rate} &= 1 - \text{Accuracy} \\ &= \frac{FP + FN}{P + N}\end{aligned}$$

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Sensitivity:**

- **Specificity:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Sensitivity:**

- True positive rate.

$$\text{Sensitivity} = \frac{TP}{P}$$

- **Specificity:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Sensitivity:**

- True positive rate.

$$\text{Sensitivity} = \frac{TP}{P}$$

- **Specificity:**

- True negative rate.

$$\text{Specificity} = \frac{TN}{N}$$

		<i>Predicted Class</i>		<i>Total</i>
		C_1	$\neg C_1$	
<i>True Class</i>	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
<i>Total</i>		P'	N'	P+N

- **Precision:**

- **Recall:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Precision:**

- Measure of exactness.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Precision:**

- Measure of exactness.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:**

- Measure of completeness.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **Precision:**

- Measure of exactness.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:**

- Measure of completeness.

$$\begin{aligned}\text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}} \\ &= \text{Sensitivity}\end{aligned}$$

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- F_β Measure:

- F_1 Measure:

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **F_β Measure:**

- Combining precision and recall.
- Gives β -times more weight to precision.
- $\beta > 1$: Minimize false positives.
- $\beta < 1$: Minimize false negatives.

$$F_\beta = \frac{(1 + \beta^2) \times \text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

- **F_1 Measure:**

		Predicted Class		Total
		C_1	$\neg C_1$	
True Class	C_1	TP	FN	P
	$\neg C_1$	FP	TN	N
Total		P'	N'	P+N

- **F_β Measure:**

- Combining precision and recall.
- Gives β -times more weight to precision.
- $\beta > 1$: Minimize false positives.
- $\beta < 1$: Minimize false negatives.

$$F_\beta = \frac{(1 + \beta^2) \times \text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

- **F_1 Measure:**

- Harmonic mean between the measures.
- Equal weight to both measures.

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	TP	FN	P
	$\neg$ Cat	FP	TN	N
<i>Total</i>		P'	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	FN	P
	$\neg$ Cat	FP	TN	N
<i>Total</i>		P'	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	FN	P
	$\neg$ Cat	FP	TN	N
<i>Total</i>		P'	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	2	5
	$\neg$ Cat	FP	TN	N
<i>Total</i>		P'	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	2	5
	$\neg$ Cat	FP	TN	N
<i>Total</i>		P'	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	2	5
	$\neg$ Cat	1	TN	N
<i>Total</i>		4	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	2	5
	$\neg$ Cat	1	TN	N
<i>Total</i>		4	N'	P+N

⁵We want to evaluate the classification model with regard to the class *Cat*

- Evaluation results of a classification model:

<i>True Class</i>	Cat	Dog	Fox	Cat	Cat	Hen	Cat	Cat	Dog	Fox
<i>Predicted Class</i>	Cat	Dog	Cat	Fox	Cat	Dog	Cat	Dog	Fox	Fox

- Resulting confusion matrix⁵:

		<i>Predicted Class</i>		
		Cat	$\neg$ Cat	<i>Total</i>
<i>True Class</i>	Cat	3	2	5
	$\neg$ Cat	1	4	5
<i>Total</i>		4	6	10

⁵We want to evaluate the classification model with regard to the class *Cat*

		<i>Predicted Class</i>		<i>Total</i>
		Cat	¬Cat	
<i>True Class</i>	Cat	3	2	5
	¬Cat	1	4	5
<i>Total</i>		4	6	10

Calculations:

$$\text{Accuracy} = \frac{TP + TN}{P + N}$$
$$=$$

Results:

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3 (TP)	2	5 (P)
	¬Cat	1	4 (TN)	5 (N)
Total		4	6	10

Calculations:

$$\begin{aligned}\text{Accuracy} &= \frac{TP + TN}{P + N} \\ &= \frac{3 + 4}{5 + 5} = \frac{7}{10} = 70\%\end{aligned}$$

Results:

$$\text{Accuracy} = 70\%$$

		<i>Predicted Class</i>		<i>Total</i>
		Cat	¬Cat	
<i>True Class</i>	Cat	3	2	5
	¬Cat	1	4	5
<i>Total</i>		4	6	10

Calculations:

$$\text{Error Rate} = \frac{\text{FP} + \text{FN}}{\text{P} + \text{N}}$$
$$=$$

Results:

$$\text{Accuracy} = 70\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3	2 (FN)	5 (P)
	¬Cat	1 (FP)	4	5 (N)
Total		4	6	10

Calculations:

$$\begin{aligned}\text{Error Rate} &= \frac{\text{FP} + \text{FN}}{\text{P} + \text{N}} \\ &= \frac{1 + 2}{5 + 5} = \frac{3}{10} = 30\%\end{aligned}$$

Results:

$$\begin{aligned}\text{Accuracy} &= 70\% \\ \text{Error Rate} &= 30\%\end{aligned}$$

		<i>Predicted Class</i>		<i>Total</i>
		Cat	¬Cat	
<i>True Class</i>	Cat	3	2	5
	¬Cat	1	4	5
<i>Total</i>		4	6	10

Calculations:

$$\text{Sensitivity} = \frac{TP}{P}$$
$$=$$

Results:

$$\text{Accuracy} = 70\%$$
$$\text{Error Rate} = 30\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3 (TP)	2	5 (P)
	¬Cat	1	4	5
Total		4	6	10

Calculations:

$$\begin{aligned}\text{Sensitivity} &= \frac{TP}{P} \\ &= \frac{3}{5} = 60\%\end{aligned}$$

Results:

$$\begin{aligned}\text{Accuracy} &= 70\% \\ \text{Error Rate} &= 30\% \\ \text{Sensitivity} &= 60\%\end{aligned}$$

		<i>Predicted Class</i>		<i>Total</i>
		Cat	¬Cat	
<i>True Class</i>	Cat	3	2	5
	¬Cat	1	4	5
<i>Total</i>		4	6	10

Calculations:

$$\text{Specificity} = \frac{TN}{N}$$
$$=$$

Results:

$$\text{Accuracy} = 70\%$$

$$\text{Error Rate} = 30\%$$

$$\text{Sensitivity} = 60\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3	2	5
	¬Cat	1	4 (TN)	5 (N)
Total		4	6	10

Calculations:

$$\begin{aligned}\text{Specificity} &= \frac{\text{TN}}{N} \\ &= \frac{4}{5} = 80\%\end{aligned}$$

Results:

$$\begin{aligned}\text{Accuracy} &= 70\% \\ \text{Error Rate} &= 30\% \\ \text{Sensitivity} &= 60\% \\ \text{Specificity} &= 80\%\end{aligned}$$

		<i>Predicted Class</i>		<i>Total</i>
		Cat	¬Cat	
<i>True Class</i>	Cat	3	2	5
	¬Cat	1	4	5
<i>Total</i>		4	6	10

Calculations:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$
$$=$$

Results:

$$\text{Accuracy} = 70\%$$

$$\text{Error Rate} = 30\%$$

$$\text{Sensitivity} = 60\%$$

$$\text{Specificity} = 80\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3 (TP)	2	5
	¬Cat	1 (FP)	4	5
Total		4	6	10

Calculations:

$$\begin{aligned}\text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}} \\ &= \frac{3}{3 + 1} = \frac{3}{4} = 75\%\end{aligned}$$

Results:

$$\begin{aligned}\text{Accuracy} &= 70\% \\ \text{Error Rate} &= 30\% \\ \text{Sensitivity} &= 60\% \\ \text{Specificity} &= 80\% \\ \text{Precision} &= 75\%\end{aligned}$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3	2	5
	¬Cat	1	4	5
Total		4	6	10

Calculations:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$
$$=$$

Results:

$$\text{Accuracy} = 70\%$$

$$\text{Precision} = 75\%$$

$$\text{Error Rate} = 30\%$$

$$\text{Sensitivity} = 60\%$$

$$\text{Specificity} = 80\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3 (TP)	2 (FN)	5
	¬Cat	1	4	5
	Total	4	6	10

Calculations:

$$\begin{aligned}\text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}} \\ &= \frac{3}{3 + 2} = \frac{3}{5} = 60\%\end{aligned}$$

Results:

$$\text{Accuracy} = 70\%$$

$$\text{Precision} = 75\%$$

$$\text{Error Rate} = 30\%$$

$$\text{Recall} = 60\%$$

$$\text{Sensitivity} = 60\%$$

$$\text{Specificity} = 80\%$$

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3	2	5
	¬Cat	1	4	5
Total		4	6	10

Calculations:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
$$=$$

Results:

Accuracy = 70%

Precision = 75%

Error Rate = 30%

Recall = 60%

Sensitivity = 60%

Specificity = 80%

		Predicted Class		Total
		Cat	¬Cat	
True Class	Cat	3	2	5
	¬Cat	1	4	5
Total		4	6	10

Calculations:

$$\begin{aligned} F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \cdot 0.75 \cdot 0.6}{0.75 + 0.6} \approx 0.6667 \approx 67\% \end{aligned}$$

Results:

Accuracy = 70%

Error Rate = 30%

Sensitivity = 60%

Specificity = 80%

Precision = 75%

Recall = 60%

$F_1 \approx 67\%$

Model Evaluation

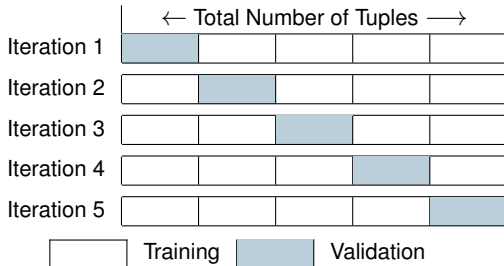
Evaluation Strategies

- **We will take a look at two types of evaluation strategies:**
 - **Methods to split data into training and test sets:**
 1. Holdout method
 2. Cross validation
 - **Methods to compare classification models and their settings:**
 1. Receiver Operating Characteristics (ROC) curve
- **Of course, there are many many more evaluation strategies**

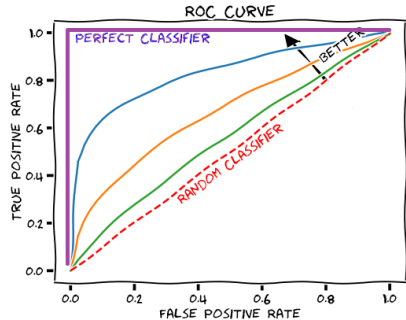
- **Easy way to split a dataset: The Holdout Method**
 - Randomly assign tuples into two independent sets:
 - **Training set** (E.g., $2/3$) for model construction.
 - **Test set** (E.g., $1/3$) for accuracy estimation.
 - Random sampling: a variation of holdout that repeats holdout k times.
 - Create an average accuracy over all experiments.

- **More robust than holdout method:**
Cross Validation
- **In this case: k -fold cross validation**
 - Randomly partition the data into k mutually exclusive subsets (folds).
 - At each iteration, use one fold as test set and the others as training set.
 - Average accuracy of all iterations.

Example: k -fold cross validation with $k = 5$



- **Receiver Operating Characteristics (ROC) curve:**
 - Visualizes the performance of a classification model:
 - Shows the performance of a model at different settings/thresholds.
 - Plots the True Positive Rate (TPR) against the False Positive Rate (FPR).
 - Shows the trade-off between sensitivity and specificity.
 - The closer the curve is to the top-left corner, the better the model.
⇒ **The area under the ROC curve (AUC) is a measure of the model's accuracy.**



Ensemble Methods

Ensemble Method

An *ensemble method* creates a composite model that consists of several models to form one model.

- **Basic Idea:**

- Use multiple models to improve classification accuracy.
- The final prediction is made by combining the predictions of all models.

- **Popular methods:**

- Bagging
- Boosting
 - Popular algorithm: AdaBoost
 - ⇒ More on that in the appendix. ⁶
- Random Forest
 - Algorithm specialized on decision trees
 - ⇒ More on that in the appendix. ⁷

⁶Not part of the exam.

⁷Not part of the exam.

- **Basic Idea:**
 - Multiple models classify the same tuple.
 - The bagged classifier collects results.
 - The class with the **most votes** is returned as the final prediction.

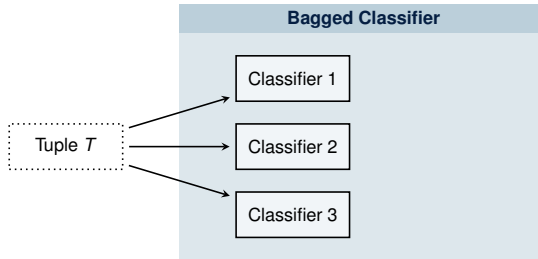
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.

Tuple T

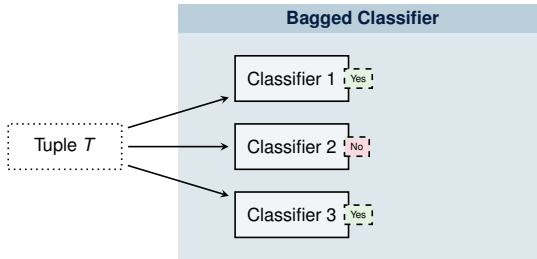
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.



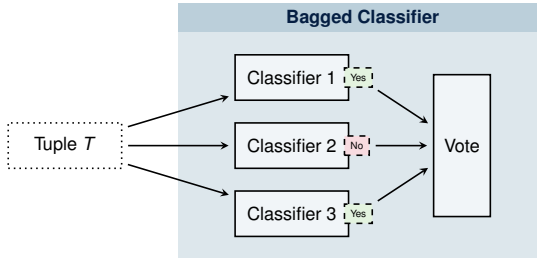
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.



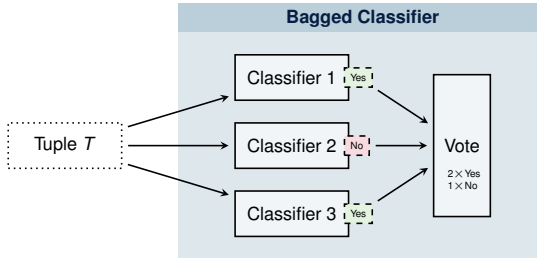
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.



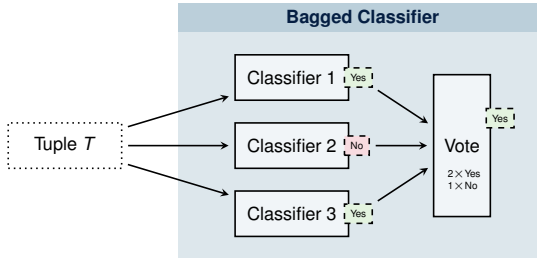
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.



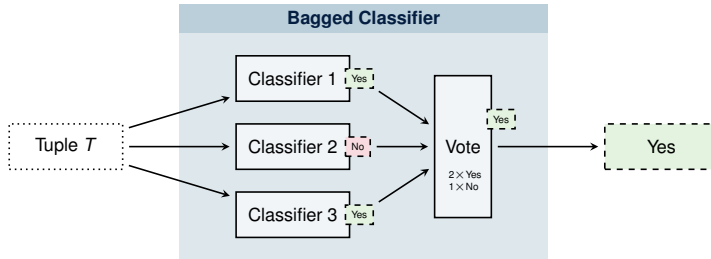
- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.



- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.

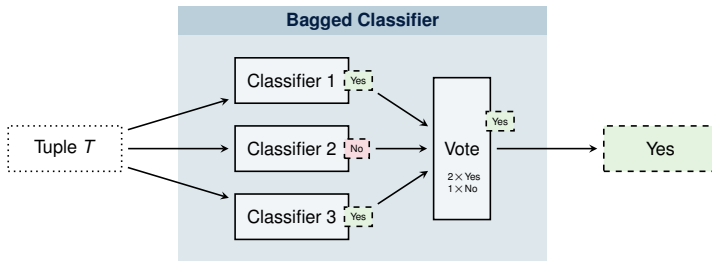


- **Basic Idea:**

- Multiple models classify the same tuple.
- The bagged classifier collects results.
- The class with the **most votes** is returned as the final prediction.

- **Continuous-valued Attributes:**

- Multiple models predict the same tuple.
- The bagged regressor collects results.
- The final prediction is the **average** of all predictions.



- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.

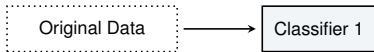
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.

Original Data

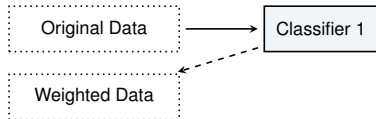
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.



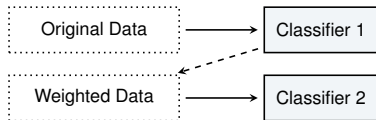
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.



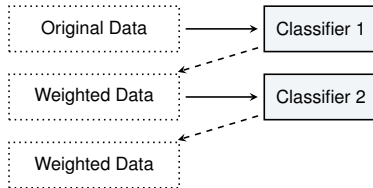
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.



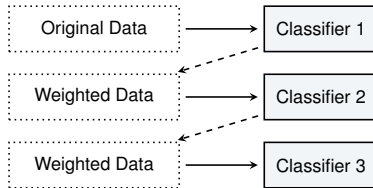
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.



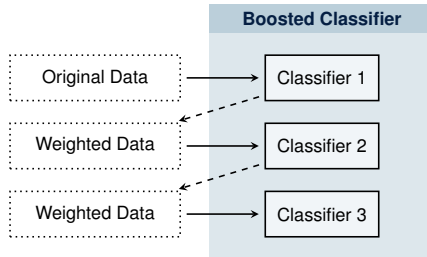
- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.



- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.

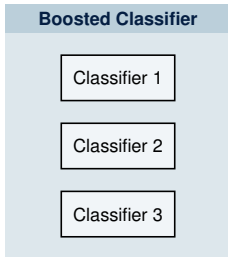


- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.

- **Predictions:**

- All models classify the same tuple.
- The boosted classifier collects results.
- The weight of each classifier's vote is a function of its accuracy.

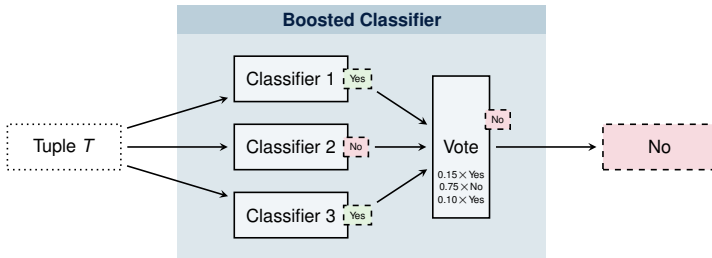


- **Training:**

- Classifiers are iteratively learned.
- After a classifier is learned, the weights of the training tuples are updated.
⇒ The next classifier pays more attention to the misclassified tuples.

- **Predictions:**

- All models classify the same tuple.
- The boosted classifier collects results.
- The weight of each classifier's vote is a function of its accuracy.



Summary

- **Classification:**
 - A form of data analysis that extracts models describing important data classes.
- **Effective and scalable methods:**
 - Decision-tree induction, rule-based classification, and naïve Bayesian classification.
- **Evaluation metrics:**
 - Accuracy, sensitivity, specificity, precision, recall, and F_β -measure.
- **Evaluation strategies:**
 - Holdout, cross-validation, and ROC-curve analysis.
- **Ensemble methods:**
 - Boosting and Bagging.

Any questions about this chapter?

Ask them now or ask them later in our forum:



 https://www.studon.fau.de/studon/goto.php?target=1code_kh3JHJdN

Appendix

Appendix

Decision Tree Induction

Data:

- Training dataset D containing tuples with their associated class labels;
- `attribute_list`, the set of candidate attributes;
- `attribute_selection_method`, a method to determine the splitting criterion that “best” partitions the data tuples into individual classes. The criterion consists of a `splitting_attribute`, and possibly, either a `split_point` or `splitting_subset`.

Result: A decision tree.

```
1 create a node  $N$ ;  
2 if tuples in  $D$  are all of the same class  $C$  then  
3   | return  $N$  as a leaf node labeled with the class  $C$ ;  
4 if attribute_list is empty then  
5   | /* Majority voting  
6   | majority_class  $\leftarrow$  determine majority class in  $D$ ;  
   | return  $N$  as a leaf node labeled with majority_class;
```

```
/* apply attribute_selection_method to find the  
   “best” splitting_criterion
```

```
7 splitting_criterion  $\leftarrow$  attribute_selection_method( $D$ ,  
   attribute_list);  
8 label node  $N$  with splitting_criterion;  
9 if (splitting_attribute is discrete-valued and multiway splits  
   allowed) or attribute value has only one unique value then  
10  | // remove splitting_attribute  
   | attribute_list  $\leftarrow$  attribute_list - splitting_attribute;  
11 foreach outcome  $j$  of splitting_criterion do  
12  | /* partition the tuples and grow subtrees for  
   | each partition */  
13  |  $D_j \leftarrow$  partition  $D$  to satisfy outcome  $j$ ;  
14  | if  $D_j$  is empty then  
   | | attach a leaf labeled with the majority class in  $D$  to node  $N$ ;  
15  | else  
   | | attach the node return by build_decision_tree( $D_j$ ,  
   | attribute_list) to node  $N$ ;  
16  |  
17 return  $N$ ;  
*/
```

Algorithm 1: `build_decision_tree`. Generate a decision tree from training tuples in data partition D .

- **CHAID:**
 - A popular decision tree algorithm, measure based on χ^2 test for independence.
- **C-SEP:**
 - Performs better than Information Gain and Gini Index in certain cases.
- **G-statistic:**
 - Has a close approximation to χ^2 distribution.
- **MDL (Minimal Description Length) principle:**
 - I.e. the simplest solution is preferred.
 - The best tree is the one that requires the fewest number of bits to both (1) encode the tree and (2) encode the exceptions to the tree.
- **Multivariate splits:**
 - Partitioning based on multiple variable combinations.
 - CART: finds multivariate splits based on a linear combination of attributes.
- **Which Attribute Selection Method is the best?**
 - Most give good results, none is significantly superior to others.

Appendix

Evaluation: Bootstrap and Statistical Significance

Bootstrap samples training data uniformly with replacement.

Several bootstrap methods exists, yet a common one is .632 bootstrap.

- Data set with d tuples is sampled d times - uniformly with replacement.
- Uniformly = every tuple has the same probability ($\frac{1}{d}$) for selection.
- With replacement = High chance a tuple is selected more than once.
- Not selected tuples will form the test set.
- Probability of not being chosen is $1 - \frac{1}{d}$. Selecting d times: $(1 - \frac{1}{d})^d$.
With a large data set it approaches $e^{-1} = 0.368$.
- Thus, on average 63.2% of tuples are selected as the training set.
- Sampling procedure is repeated k times.
Calculate accuracy in every iteration as follows:

$$\text{Acc}(M) = \frac{1}{k} \sum_{i=1}^k 0.632 \cdot \text{Acc}(M_i)_{\text{test_set}} + 0.368 \cdot \text{Acc}(M_i)_{\text{train_set}}.$$

- **Suppose we have 2 classifiers, M_1 and M_2 , which one is better?**
- **Use 10-fold cross-validation to obtain $\overline{\text{err}}(M_1)$ and $\overline{\text{err}}(M_2)$.**
 - Recall: error rate is $1 - \text{accuracy}(M)$.
- **Mean error rates:**
 - Just estimates of error on the true population of future data cases.
- **What if the difference between the 2 error rates is just attributed to chance?**
 - Use a test of statistical significance.
 - Obtain confidence limits for our error estimates.

- **Perform k -fold cross-validation** with $k = 10$.
- **Assume samples follow a t -distribution** with $k - 1$ degrees of freedom.
- **Use t -test**
 - Student's t -test.
- **Null hypothesis:**
 - M_1 and M_2 are the same.
- **If we can reject the null hypothesis, then**
 - Conclude that difference between M_1 and M_2 is statistically significant.
 - Obtain confidence limits for our error estimates.

- **If only one test set available: pairwise comparison:**

- For i -th round of 10-fold cross-validation, the same cross partitioning is used to obtain $\text{err}(M_1)_i$ and $\text{err}(M_2)_i$.
- Average over 10 rounds to get $\overline{\text{err}}(M_1)$ and $\overline{\text{err}}(M_2)$.
- t -test computes t -statistic with $k - 1$ degrees of freedom:

$$t = \frac{\overline{\text{err}}(M_1) - \overline{\text{err}}(M_2)}{\frac{\text{var}(M_1 - M_2)}{\sqrt{k}}},$$

- where

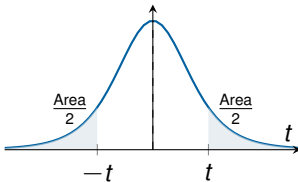
$$\text{var}(M_1 - M_2) = \frac{1}{k} \sum_{i=1}^k [\text{err}(M_1)_i - \text{err}(M_2)_i - (\overline{\text{err}}(M_1) - \overline{\text{err}}(M_2))]^2.$$

- **If two test sets available: use unpaired t -test:**

$$\text{var}(M_1 - M_2) = \sqrt{\frac{\text{var}(M_1)}{k_1} + \frac{\text{var}(M_2)}{k_2}},$$

where k_1 & k_2 are # of cross-validation samples used for M_1 & M_2 , respectively.

- Symmetrical.
- **Significance level:**
 - E.g., $\text{sig} = 0.05$ or 5% means M_1 & M_2 are significantly different for 95% of population.
- Confidence limit: $z = \frac{\text{sig}}{2}$.



df/α	Area in One Tail ¹		
	0.100	0.050	0.005
	Area in Two Tails ¹		
	0.200	0.100	0.010
1	3.078	6.314	63.657
2	1.886	2.920	9.925
3	1.638	2.353	5.841
4	1.533	2.132	4.604
5	1.476	2.015	3.707
6	1.440	1.943	3.499
7	1.415	1.895	3.355
8	1.397	1.860	3.250
9	1.372	1.833	3.169

¹Good link for a full table: https://www.hawkeslearning.com/documents/statdatasets/stat_tables.pdf

Are M_1 and M_2 significantly different?

- Compute t . Select significance level (E.g., $\text{sig} = 5\%$).
- Consult table for t -distribution:
 - t -distribution is symmetrical:
 - Typically upper % points of distribution shown.
 - Find critical value c corresponding to $k - 1$ degrees of freedom (here, 9)
 - and for confidence limit $z = \frac{\text{sig}}{2}$ (here, 0.025).
 - $\implies$ Here, critical value $c = 2.262$
- If $t > c$ or $t < -c$, then t value lies in rejection region:
 - **Reject null hypothesis** that mean error rates of M_1 and M_2 are equal.
 - Conclude: **statistically significant difference** between M_1 and M_2 .
- Otherwise, conclude that any difference is chance.

Appendix

Ensemble Methods: AdaBoost and Random Forests

- **Given a data set D of d class-labeled tuples:** $(x_1, y_1), \dots, (x_d, y_d)$ with $y_d \in Y = \{1, \dots, c\}$.
- **Initialize empty lists to hold information per classifier:** $\mathbf{w}, \beta, \mathbf{M} \leftarrow$ empty list.
- **Initialize weights for first classifier to hold same probability for each tuple:** $w_j^1 \leftarrow \frac{1}{d}$
- **Generate K classifiers in K iterations. At iteration k ,**
 1. Calculate "normalized" weights: $\mathbf{p}^k = \frac{\mathbf{w}^k}{\sum_{j=1}^d w_j^k}$
 2. Sample dataset with replacement according to $\mathbf{p}^k$ to form training set D_k .
 3. Derive classification model M_k from D_k .
 4. Calculate error ε_k by using D_k as a test set as follows: $\varepsilon_k = \sum_{j=1}^d p_j^k \cdot \text{err}(M_k, x_j, y_j)$,
where the *misclassification error* $\text{err}(M_k, x_j, y_j)$ returns 1 if $M_k(x_j) \neq y_j$, otherwise it returns 0.
 5. If $\text{error}(M_k) > 0.5$: Abandon this classifier and go back to step 1.
 6. Calculate $\beta_k = \frac{\varepsilon_k}{1 - \varepsilon_k}$.
 7. Update weights for the next iteration: $w_j^{k+1} = w_j^k \beta_k^{1 - \text{err}(M_k, x_j, y_j)}$. *If a tuple is misclassified, its weight remains the same, otherwise it is decreased.* Misclassified tuple weights are increased relatively.
 8. Add $\mathbf{w}^{k+1}$, M_k , and β_k to their respective lists.

- **Initialize weight of each class to zero.**
- **For each classifier i in k classifiers:**
 1. Calculate the weight of this classifier's vote: $w_i = \log(\frac{1}{\beta_i})$.
 2. Get class prediction c for (single) tuple x from current weak classifier M_i : $c = M_i(x)$.
 3. Add w_i to weight for class c .
- **Return predicted class with the largest weight.**
- Mathematically, this can be formulated as:

$$M(x) = \arg \max_{y \in Y} \sum_{i=1}^k (\log \frac{1}{\beta_i}) M_i(x)$$

⁹Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997. DOI: 10.1006/jcss.1997.1504 [Online]. Available: <https://doi.org/10.1006/jcss.1997.1504> Algorithm AdaBoost.M1 on p. 131.

- Ensemble method consisting only of decision trees where each tree has been generated using random selection of attributes at each node.
- Classification: Each tree votes and the most popular class is returned.
- **Two methods to construct random forests:** (each builds k trees)
 1. Forest-RI (random input selection):
 - Random sampling with replacement to obtain training data from D .
 - Set F as the number of attributes to determine split at each node. F is smaller than the number of available attributes.
 - Construct decision tree M_i by randomly select candidates at each node. Use CART to grow tree to maximum size without pruning.
 2. Forest-RC: Similar to Forest-RI but new attributes (features) are generated by linear combinations of existing attributes to reduce correlation between individual classifiers. At each node, attributes are randomly selected.
- **Comparable in accuracy to AdaBoost, but more robust to errors and outliers.**
- **Insensitive to the number of attributes selected for consideration at each split, and faster than bagging or boosting.**

¹²Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997. DOI: 10.1006/jcss.1997.1504 [Online]. Available: <https://doi.org/10.1006/jcss.1997.1504> Algorithm AdaBoost.M1 on p. 131.

Class-Imbalanced Data

Class-Imbalanced Data refers to data where the main class of interest (positive labeled) is only represented by a small number of tuples. E.g., medical diagnosis and fraud detection.

- Problem because traditional methods assume *equality between classes*, i. e. a balanced distribution of classes and equal error costs.
- **Typical methods for imbalanced data in binary classification:**
 1. **Undersampling/Oversampling:** Changes distribution of tuples in training data.
 - *Undersampling:* Randomly eliminate tuples from negative class.
 - *Oversampling:* Re-samples data from positive class.
For instance, method SMOTE generates synthetic data that is similar to existing data using nearest neighbor.
 2. **Threshold-moving:** Moves the decision threshold, t , so that the rare-class tuples are easier to classify, and hence, less chance of costly false-negative errors. Works when class returns a probability.
 3. **Ensemble techniques.**

Threshold-moving and ensemble methods work well on extremely imbalanced data.

- **Still difficult on multi-class tasks.**

¹³L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324 [Online]. Available: <https://doi.org/10.1023/A:1010933404324>